# Dokumentacja końcowa – MED

Piotr Siciński
10.01.2026

## Opis zadania

**Cel projektu**

Celem projektu było zaimplementowanie i zbadanie algorytmu do wielopoziomowego wyszukiwania reguł asocjacyjnych, opartego na artykule:

*Jiawei Han and Yongjian Fu, "Mining multiple-level association rules in large databases," in IEEE Transactions on Knowledge and Data Engineering, vol. 11, no. 5, pp. 798-805, Sept.-Oct. 1999, doi: 10.1109/69.806937*.

**Założenia dotyczące algorytmu**

1. Algorytm przetwarza dane na wielu poziomach abstrakcji, wykorzystując taksonomię pojęć od poziomu ogólnego do szczegółowego.
2. Implementacja wykorzystuje podejście "od góry do dołu", gdzie:
   - Najpierw wyznaczane są częste elementy na najwyższym poziomie taksonomii.
   - Następnie algorytm rekurencyjnie zagłębia się w potomków częstych elementów na niższych poziomach. Elementy rzadkie są przycinane wraz z ich poddrzewami, skracając czas obliczeń.
3. Algorytm bazuje na klasycznym Apriori:
   - Generowanie k-zbiorów z (k-1)-zbiorów
   - Krok łączenia (Join) redukowania (Prune) zgodnie z algorytmem Apriori
   - Generowanie reguł asocjacyjnych z częstych elementów

**Założenia implementacyjne**

1. Do reprezentacji wystąpień atrybutów w transakcjach wybrano listy wystąpień (TID list) zamiast oryginalnego mechanizmu w Apriori. Z uwagi na zmienną gęstość wystąpień atrybutów w zależności od rozważanego poziomu (ogólne pojęcia będą występować częściej niż rzadkie), wykorzystano RoaringBitmap ze względu na swoją właściwość adaptacji. Oferuje kompresję bitmapy do struktury rzadkiej przy niskiej gęstości wystąpień

(tak jak TID listy). A przy wysokiej gęstości pozostaje bitmapą, umożliwiając błyskawiczne operacje logiczne na wielu bitach jednocześnie.

2. Język implementacji – Rust. Wybrany ze względu na:
   - wysoką wydajnością (porównywalna z C/C++),
   - bezkosztowe abstrakcje,
   - bezpieczeństwo pamięci bez garbage collector,
   - bardzo dobrą obsługą zaawansowanych struktur danych.
3. Ograniczenia:
   - Maksymalna liczba atrybutów na wszystkich poziomach taksonomii: $2^{32}$.
   - Maksymalna liczba transakcji nie jest określona, zależy od liczby występujących atrybutów oraz częstości ich występowania, a ograniczeniem jest pojemność pamięci operacyjnej komputera. Program działający na zbiorze około 2 milionów transakcji i 10 tysięcy atrybutów potrzebuje 160 MB wolnej pamięci operacyjnej.
   - Taksonomia atrybutów musi być reprezentowana jako drzewo doskonałe – wszystkie liście (atrybuty występujące w transakcjach) znajdują się taj samej głębokości. Takie założenie przyjęli także autorzy metody.
   - Program jest jednowątkowy; nie wykorzystuje zrównoleglania, aby wydatniej przedstawić algorytmiczne korzyści wielopoziomowego filtrowania atrybutów.

# Opis projektu implementacji algorytmu

## Architektura systemu

**Moduł Repository**

Odpowiedzialny za:

- Wczytywanie i parsowanie danych wejściowych (transakcje, taksonomia)
- Budowę hierarchicznej struktury drzewa taksonomii
- Propagację bitmap od liści do korzenia
- Zarządzanie strukturą drzewa podczas przetwarzania

```
pub struct Repository {
    pub arena: Arena<NodeData>,           // Drzewo taksonomii, Arena – implementacja drzeawa
z biblioteki indextree
    pub root: NodeId,                     // Korzeń drzewa
    pub node_by_id: HashMap<u32, NodeId>, // Mapowanie ID -> węzeł
    pub transaction_count: u32,           // Liczba transakcji
}
```

```
pub struct NodeData {
    pub id: u32,                  // Unikalny identyfikator atrybutu
    pub name: String,             // Nazwa atrybutu
    pub bitmap: RoaringBitmap,    // Transakcje zawierające element
}
```

**Moduł Apriori**

Implementuje rdzeń algorytmu wielopoziomowego wyszukiwania reguł asocjacyjnych. Uruchamia kolejno Apriori dla atrybutów z kolejnych poziomów taksonomii, filtrując rzadkie atrybuty.

```
pub struct Apriori {
    pub repository: Repository,
    pub level_configs: HashMap<i32, LevelConfig>,
}
```

`level_configs` pozwala wybrać, na których poziomach ma zostać wykonane filtrowanie rzadkich atrybutów i generowanie reguł. Implementacja może zostać wykorzystana jako standardowe Apriori, jeśli dokona się wyszukiwania reguł tylko na najniższym poziomie.

```
pub struct LevelConfig {
    pub support: f64,               // Min. wsparcie (0.0-1.0)
    pub confidence: f64,     // Min. ufność (0.0-1.0)
    pub gen_rules: bool,     // Czy generować reguły
}
```

W artykule zawsze liczone są reguły na wszystkich poziomach oraz filtrowane są rzadkie atrybuty. Aby zapewnić większą elastyczność pod kątem testów, w tej implementacji można samem decydować, na których poziomach nastąpi uruchamianie Apriori, a na których tylko filtrowanie atrybutów (pole gen_rules).

**Moduł Experiments**

Zawiera kod do przeprowadzania eksperymentów, badając czas wykonania i otrzymywane wyniki przez algorytm w poszczególnych scenariuszach. Wyniki zapisuje w plikach w formacie JSON, które następnie są ładowane do skryptu w języku Python tworzącego odpowiednie wykresy.

## Implementacja Repository

Kroki:

1. Parsowanie pliku taksonomii (format: `child parent` w każdej linii)
2. Budowa struktury drzewiastej z węzłów NodeData
3. Walidacja: wszystkie liście muszą być na najniższym poziomie (jednoznaczność)

4. Jeśli brak wspólnego korzenia, tworzony jest sztuczny węzeł-korzeń
5. Parsowanie pliku transakcji (format: `item1 item2 item3 ... `)
6. Aktualizacja bitmap w liściach poprzez `node_by_name`
7. Propagacja bitmap w górę drzewa dzięki przechodzeniu DFS. Bitmapa węzła rodzica powstaje jako union bitmap dzieci, co odpowiada hierarchizacji.

## Implementacja Apriori

Kroki:

1. Konwersja ujemnych wartości `level` z LevelConfig na wartości bezwzględne
2. Sortowanie `level_configs` rosnąco według `level`
3. Tworzenie hashsetów węzłów dla każdego poziomu – dzieci węzłów z hashsetu wyższego poziomu. Daje to szybki dostęp do węzłów na danym poziomie
4. Poziom 0 (korzeń) nie jest rozważany

Dla każdego poziomu (od 1 do max_level):

jeśli istnieje LevelConfig dla tego poziomu:

1. Filtracja 1-itemsetów:
   - oblicz support dla każdego węzła na poziomie
   - usuń węzły z supportem < min_support (wraz z poddrzewami)
     Jest to odpowiednik akcji z artykułu usuwania rzadkich atrybutów z transakcji
   - zaktualizuj hashset dla tego poziomu

2. Generacja frequent itemsets - wywołaj freqsets_from_level(level, support)

3. Generacja reguł:
   - wywołaj generate_rules(frequent_itemsets, confidence)
   - zapisz reguły dla tego poziomu

4. Przygotowanie następnego poziomu - z hashsetu bieżącego poziomu utwórz hashset dzieci

w przeciwnym razie:

nie twórz reguł i nie usuwaj żadnych atrybutów

zwróć hashmapę: poziom -> lista reguł

# Opis implementacji programu

## Repository

- Parsowanie pliku taksonomii
- Dynamiczne tworzenie węzłów przy odczycie pliku opisującego taksonomię
- Obsługa wielu korzeni - automatyczne tworzenie sztucznego korzenia ROOT
- Wykorzystanie `indextree::Arena` dla efektywnego zarządzania pamięcią
- Parsowanie pliku transakcji

**Wczytywanie transakcji**

```
fn load_transactions(
    arena: &mut Arena<NodeData>,
    node_by_name: &HashMap<String, NodeId>,
    transactions_path: &str,
) -> Result<u32, Box<dyn std::error::Error>>
```

- Pominięcie elementów spoza taksonomii
- Usuwanie komentarzy (linie zaczynające się od '@')
- Każda transakcja otrzymuje unikalny ID (rosnący licznik)

**Propagacja bitmap**

```
fn propagate_bitmaps(arena: &mut Arena<NodeData>, node_id: NodeId)
```

Algorytm post-order traversal (DFS):

1. Rekurencyjnie propaguj dla wszystkich dzieci
2. Utwórz & bitmap wszystkich dzieci
3. Przypisz do bieżącego węzła

## Apriori

**Główna pętla**

```
pub fn mine(&mut self) -> HashMap<usize, Vec<Rule>>
```

Implementacja poziomowego usuwania rzadkich zbiorów oraz generowania reguł:

- Zbiór węzłów na bieżącym poziomie powstaje jako dzieci węzłów z poziomu wyżej
- Przycinanie przez  remove_subtree dla odrzuconych węzłów
- get_children - przejście do następnego poziomu tylko dla częstych węzłów
- Warunek stopu: current_nodes.is_empty()

**Łączenie i przycinanie kandydatów - generowanie kandydatów**

```
fn join_step(&self, itemsets: &[Itemset]) -> Vec<Itemset>
```

Implementacja zgodna z oryginalnym algorytmem Apriori:

- Porównanie prefixów (wszystkie oprócz ostatniego elementu)

```
fn prune_step(&self, itemmap: &HashMap<Itemset, u32>, candidates: &[Itemset]) -> Vec<Itemset>
```

**Obliczanie wsparcia - przecięcie bitmap**

```
fn calculate_support(&self, itemset: &Itemset) -> u32
```

Implementacja wykorzystująca RoaringBitmap:

- Optymalizacja dla 1-itemsetów: bezpośredni zwrot długości bitmap
- Dla większych: pierwsza bitmapa jest kopiowana i przecinana operatore & z pozostałymi

**Generowanie reguł**

```
fn generate_rules(&self, itemsets: &HashMap<usize, HashMap<Itemset, u32>>, min_confidence:
f64) -> Vec<Rule>
```

1. Jednolementowe następniki: znajdowane poprzez przejście przez zbiór elementów
2. Wieloelementowe następniki: tworzone rekurencyjnie

## Testowanie poprawności

Poprawność implementacji została sprawdzona w wykorzystaniem testów jednostkowych i funkcjonalnych poszczególnych klas i modułów.

# Eksperymenty

Eksperymenty miały na celu:

1. Ocenę wrażliwości parametrów - jak progi wsparcia i ufności wpływają na czas wykonania i liczbę reguł
2. Ocenę skalowalności - jak algorytm radzi sobie ze wzrostem liczby transakcji i atrybutów
3. Porównanie z podejściem płaskim - wykazanie korzyści z hierarchicznego przetwarzania względem standardowego Apriori
4. Analiza efektywności przycinania - wpływ progów wsparcia dla elementów na wyższych poziomach na efektywność

## Zbiory danych

Wykorzystano dwa rzeczywiste zbiory danych z Instacart (platforma zakupów spożywczych online):

**Taksonomia: instacart_taxonomy.txt**

- Wspólna dla obu zbiorów danych
- 3-poziomowa (Dział, Kategoria, Produkt)
- Format: CSV (dziecko,rodzic)

**Dataset 1: instacart_trainFIM.txt**

- 131,209 transakcji, 49,677 produktów
- Transakcje zakupowe z zestawu treningowego
- Format: jedna tranzakcja w linii (identyfikatory produktów oddzielone spacjami)

**Dataset 2: instacart_priorFIM.txt**

- 3,214,874 transakcji, 39,123 produktów
- Większy zbiór danych niż train
- Identyczny format

Wybrano te zbiory danych, ponieważ stworzono je z rzeczywistych danych oraz dołączona taksonomia jest drzewem pełnym - liście (poszczególne produkty) znajdują się na tylko na najniższym poziomie drzewa, co jest wymagane do tej metody.

## Domyślna konfiguracja wielopoziomowa

Wszystkie eksperymenty (oprócz 3.2) używają domyślnej konfiguracji bazowej:

```
let mut configs = HashMap::new();
configs.insert(1, LevelConfig {
    support: 0.5,
    confidence: 0.2,
    gen_rules: false/true,  // zależnie od eksperymentu
});
configs.insert(2, LevelConfig {
    support: 0.08,
    confidence: 0.2,
    gen_rules: false/true,  // zależnie od eksperymentu
});
configs.insert(3, LevelConfig {
    support: 0.005,
    confidence: 0.2,
    gen_rules: true,
});
```

Uzasadnienie:

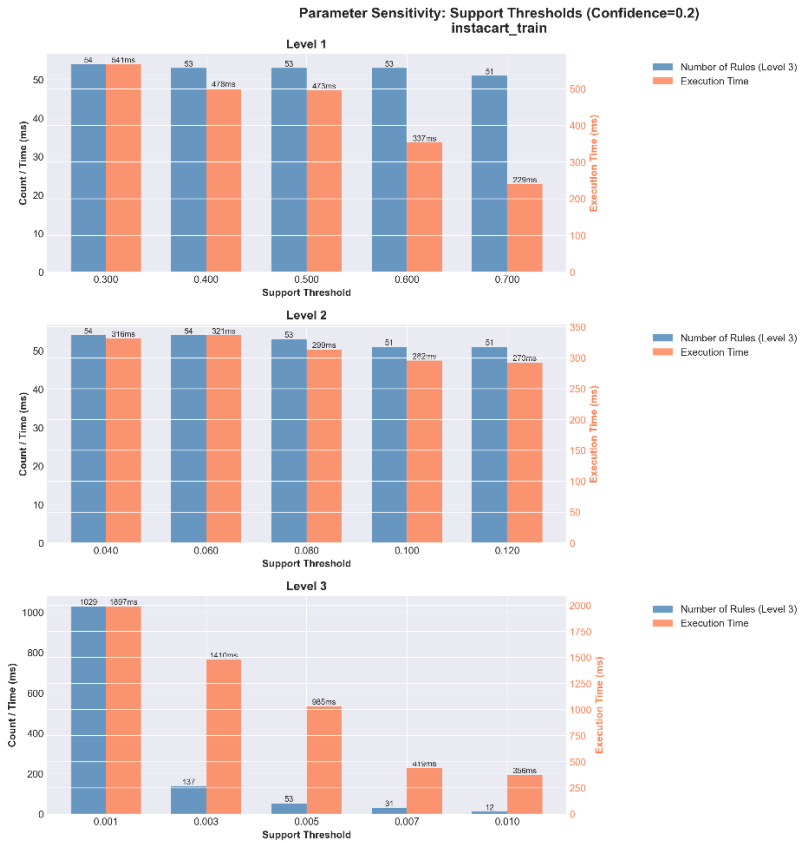
- Poziom 1 (0.5): Wysokie wsparcie dla ogólnych kategorii (działy)
- Poziom 2 (0.08): Średnie wsparcie dla kategorii
- Poziom 3 (0.005): Niskie wsparcie dla konkretnych produktów
- Progresywne zmniejszanie jest zgodne z naturalną strukturę danych – im szersza kategoria, tym więcej wystąpień

Każdy eksperyment uruchamiano 5 razy, a wyniki są uśredniano w celu zmniejszenia wpływu zaburzonych próbek.

## Eksperyment 1: Wrażliwość na wartości parametrów

**Wsparcie**

Każdy test zmienia tylko próg na jednym poziomie hierarchii, pozostałe są domyślne. Reguły wyszukiwane są tylko na 3. Poziomie.




Dla zbioru train widoczny jest związek wartości parametru wsparcia z czasem działania algorytmu, jednak w różny sposób w zależności od rozważanego poziomu.

Na 1. Poziomie (najbardziej ogólne pojęcia), zwiększanie progu wsparcia dla itemsetów powoduje znaczny spadek czasu pracy algorytmu przy minimalnym zmniejszeniu liczby znajdowanych reguł. Pokazuje to największą zaletę tego hierarchicznego podejścia – filtrując ogólne kategorie, oszczędzamy pracy na niższych poziomach redukując liczbę potrzebnych obliczeń.

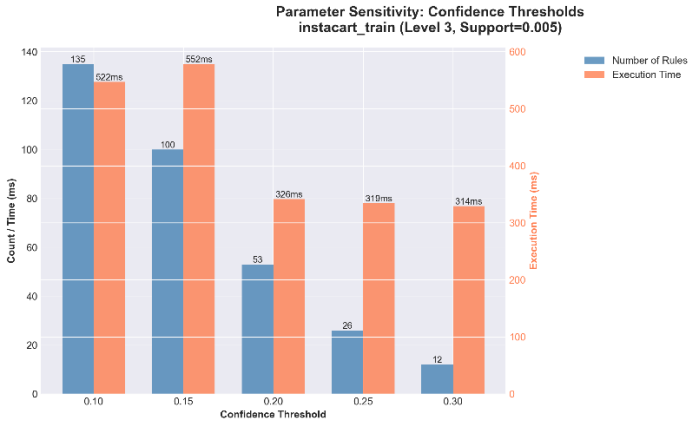
Na 2. Poziomie wpływ ten był znacznie mniejszy. Możliwe, że wartości testowe nie zostały najtrafniej dobrane, jednak ich wybór nie był prosty ze względu na brak przesłanek odnośnie ich porządanego zakresu.

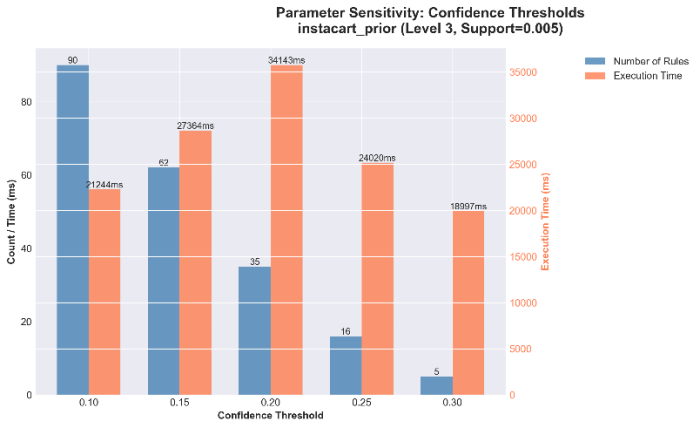
Na 3. Poziomie modyfikacja progu wsparcia bezpośrednio wiązała się zarówno z czasem wykonania, jak i liczbą znajdowanych reguł. Jest to zgodne z podstawowym mechanizmem działania Apriori.

Dla zbioru prior wyniki mają podobny charakter.

**Pewność**

Reguły wyszukiwane są tylko na 3. Poziomie. Wsparcie 0.005.

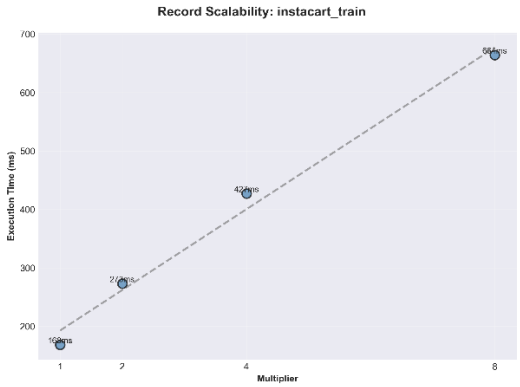


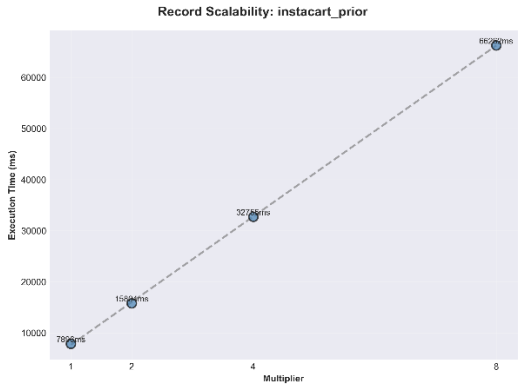




Pewność nie wpływa w istotny sposób na czas obliczeń. Zgodnie z oczekiwaniami filtruje słabe reguły, jednak to liczba itemsetów odgrywa kluczową rolę jeśli chodzi o ilość obliczeń.

## Eksperyment 2: Badanie skalowalności – liczba rekordów

Rozszerzono zbiory danych symulując większe datasety przy zachowaniu tej samej dystrybucji itemsetów.




Czas działania zaimplementowanego algorytmu jest liniowy w stosunku do rozmiaru problemu. Następuje także proporcjonalny wzrost zapotrzebowania na pamięć operacyjną.

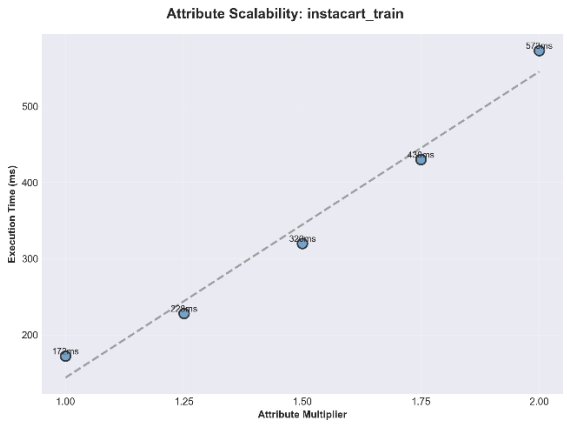
Podwojenie rozmiaru problemu skutkuje 1,4 i 2,1 krotnym wzrostem czasu obliczeń odpowiednio dla zbioru train i prior.

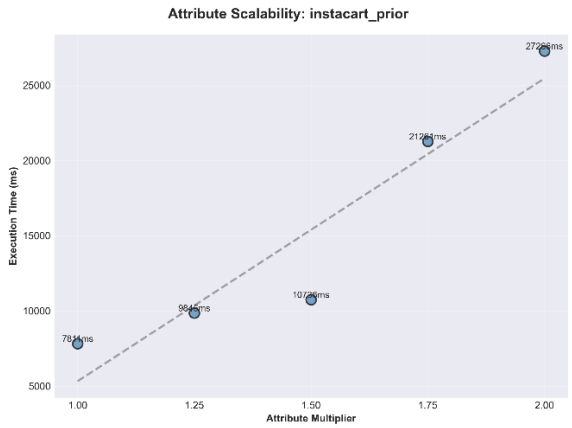
## Eksperyment 3: Badanie skalowalności – liczba atrybutów

Strategia generowania syntetycznych atrybutów:

1. Losowo wybierz 5 produktów
2. Dla każdego transaction ID w źródłowych bitmap dodaj je do nowego syntetycznego produktu z prawdopodobieństwem 0.2.
3. Przypisz syntetyczny atrybut do rodzica pierwszego z produktów
4. Powtórz `(multiplier - 1.0) * leaf_count` razy

Dzięki temu powstaje nowy atrybut, który posiada pewne cechy już obecnych w zbiorze atrybutów.




Czas działania zaimplementowanego algorytmu jest liniowy w stosunku do liczby atrybutów.

Podwojenie liczby atrybutów skutkuje około 4-krotnym wzrostem czasu obliczeń dla obu zbiorów.

## Eksperyment 4: Porównanie z podejściem płaskim

W tym eksperymencie porównano czas, jaki byłby potrzebny na znalezienie reguł z wielu poziomów korzystając z:

1. 3 niezależnych uruchomień algorytmu w trybie jednopoziomowym, co odpowiada klasycznemu Apriori,
2. Jednego uruchomienia ze wszystkimi poziomami.

Zbiór train

| Metryka | 3 uruchomienia | Wielopoziomowe uruchomienie |
|---|---|---|
| Reguły na poziomach | 2, 90, 55 | 2, 83, 53 |
| Reguły łącznie | 147 | 138 |
| Czasy uruchomień (ms) | 137, 142, 466 | - |
| Czas całkowity (ms) | 745 | 313 |
| Oszczędność czasu (%) | | **58,0%** |

Zbiór prior

| Metryka | 3 uruchomienia | Wielopoziomowe uruchomienie |
|---|---|---|
| Reguły na poziomach | 2, 73, 37 | 2, 68, 35 |
| Reguły łącznie | 112 | 105 |
| Czasy uruchomień (ms) | 16115, 15487, 34317 | - |
| Czas całkowity (ms) | 65919 | 22936 |
| Oszczędność czasu (%) | | **65,2%** |

Algorytm wielopoziomowy osiąga zbliżone wyniki do klasycznego jeśli chodzi o liczbę reguł, skracając czas pracy 2.5-krotnie.

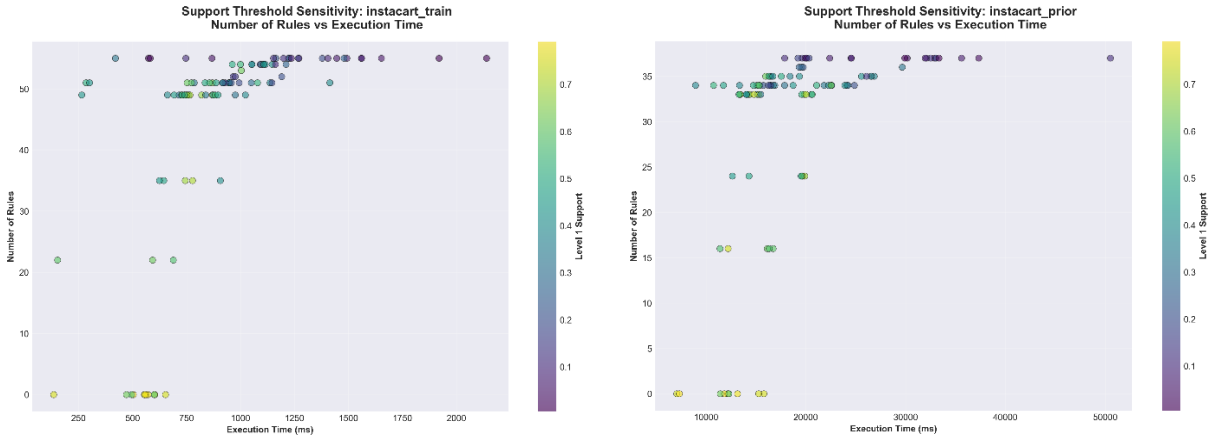
## Eksperyment 5: Analiza efektywności przycinania

Aby szerzej zbadać wpływ efektywności redukcji liczby itemsetów na czas obliczeń, przeprowadzono przemiatanie wartości wsparcia na 1. I 2. Poziomie.

Wylosowano 100 zestawów wartości wsparcia, w których:

```
level_1_support = uniform_random(0.005, 0.8)
level_2_support = uniform_random(0.005, level_1_support)
level_3_support = 0.005
```

Liczbę znalezionych reguł oraz czas wykonania naniesiono na wykres punktowy.




Przeprowadzony eksperyment pokazał, że właściwy dobór wartości wsparcia na poszczególnych poziomach jest kluczowy dla znalezienia większości reguł. Taki sam czas obliczeń mogą dawać parametry znajdujące wszystkie reguły, jak i te poważnie ograniczające ich liczbę.

Najlepsze ustawienia wartości wsparcia, które minimalizowały czas obliczeń i jednocześnie znajdowały wszystkie reguły, pozwalały na 5,4- i 2.7-krotne przyspieszenie wykonania.

# Instrukcja użycia oprogramowania

Należy zainstalować pakiet języka `Rust` wraz z menadżerem paczek `Cargo` (dostępny na większość systemów operacyjnych).

W folderze `datasets` należy umieścić plik txt transakcji oraz plik txt taksonomii w formacie opisanym w podrozdziale Eksperymenty „Zbiory danych”. W eksperymentach użyto:

https://www.philippe-fournier-viger.com/spmf/publicdatasets/instacart/instacart_priorFIM.txt

https://www.philippe-fournier-viger.com/spmf/publicdatasets/instacart/instacart_trainFIM.txt

https://www.philippe-fournier-viger.com/spmf/publicdatasets/instacart/taxonomy.txt

Aby zbudować i uruchomić oprogramowanie należy wykonać:

```
cargo build --release

// uruchomienie program main
cargo run --release --bin hqapriori

// uruchomienie eksperymentów
cargo run --release --bin experiments

// uruchomienie testów
cargo test
```

Plik main zawiera przykład użycia implementacji algorytmu.